

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a628423f96ce7e540.jsonl`  
- SHA-256: `f96899b6b6f83702c1cac21de2bb114c959c2ba72f481e036addf9386c15900e`  
- Source modified: `2026-07-02T03:02:15+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T03:00:39.192Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.

A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:

```
{  
  "title": "7 stale scratch/*.py files tracked in git, violating the repo's own scratch convention",  
  "category": "hygiene",  
  "severity": "P2",  
  "file": "scratch/test_ai_segmenter.py",  
  "line": 1,  
  "evidence": "git ls-files shows 7 tracked scratch/*.py (analyze_motion.py, clean_state.py, diagnose_rally.py, gemini_video_debugger.py, list_models.py, test_ai_segmenter.py, test_thresholds.py) added in pre-convention checkpoint commits (6b7d5dd, 0f76429, 72f972e). The repo's own rules say scratch/ is gitignored and never committed: docs/CODE_MAP.md:34 ('Gitignored + excluded from ruff/pytest. Never imported by app code'), .gitignore:71, ruff.toml:8-10 extend-exclude. These tracked files import live backend modules unlited and untested (scratch/test_ai_segmenter.py:6-7 imports backend.infrastructure.database + backend.pipeline.segmenters) and hardcoded owner-machine paths (scratch/test_ai_segmenter.py:10 r'C:\\Users\\avidu\\Videos\\TestLargeVideo.MP4'). The ~20 other scratch/*.py on disk are correctly untracked. artifacts/ is clean ? 0 tracked files, .gitignore:53 working.",  
  "proposed_fix": "git rm --cached scratch/*.py (7 files) so the directory matches the documented convention; anything still useful gets promoted to scripts/ per the CODE_MAP rule.",  
  "effort": "S",  
  "tracked_in": ""  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T03:00:50.035Z)

Already up to date.

master

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)

3. user (2026-07-02T03:00:57.633Z)

```
scratch/analyze_motion.py
scratch/clean_state.py
scratch/diagnose_rally.py
scratch/gemini_video_debugger.py
scratch/list_models.py
scratch/test_ai_segmenter.py
scratch/test_thresholds.py
---artifacts---
0
---untracked scratch py count---
55
```

4. user (2026-07-02T03:01:07.244Z)

```
52:# Local generated artifacts (review reels, eval CSVs, e2e/proof outputs) -- regenerable,
never committed
53:artifacts/
69:# Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from
71:scratch/
---ruff---
10:extend-exclude = ["scratch"]
11-
12-[lint]
13-extend-select = ["W", "B", "I"]
14-
---codemap---
34:| A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded from ruff/pytest.
Never imported by app code. |
116: - **STUDIO / STORAGE endpoints (P1/P2/P3/P4/P6/P8, 2026-07-01):** `::capabilities` `@GET
/api/capabilities` ? honest box probe; now includes a **`storage` block**
(`storage_capabilities` in `@backend/storage/capabilities.py`: local/s3/gcs/gdrive
`{id,kind,label,usable,free_gb}`, secret-free) alongside segmenters/ffmpeg/gpu/deployment.
`::list_videos` `@GET /api/videos` (the "my videos" list, P6). `::get_source` `@GET
/api/source/{id}` ? Range-capable FileResponse serving a **frame-identical annotation proxy**
(D4), source fallback, background-warmed. `::post_annotations` `@POST /api/annotations` ? writes
the verbatim annotator CSV to `output/labels/<video_id>.rallies.csv` (P8). `::create_asset`
`@POST /api/assets` (**P4**, `{uploaded_path?|source_uri?, medium_uri, sport?}`) ? stores a
video into a user-chosen medium (MD5-keyed `storage.put` for an upload?remote medium; register-
in-place for local; at-rest URI = no copy), registers it (video_id=MD5) so it shows in
`/api/videos`; **self-host posture only** (403 when `security.allowed_video_root` set); edge-
auth mirrors `/api/process`. Cost ledger: `::get_job_cost` `@GET /api/jobs/{id}/cost`,
`::get_video_cost` `@GET /api/videos/{id}/cost`. **Free-space guard (P2):**
`@backend/utils/freespace.py::require_free_bytes` -> HTTP **507** in `uploads.py::upload_init` +
the `/api/process` remote fetch-to-scratch; the WSL frame-extraction guard now **BLOCKS**
(`detectors/base.py::_wsl_frame_stage_shortfall`, ~15x source).
```

5. user (2026-07-02T03:01:07.297Z)

```
1      import os
2      import json
3      from dotenv import load_dotenv
4
5      load_dotenv()
6      from backend.infrastructure.database import Database
7      from backend.pipeline.segmenters import segmenter_registry
8
```

```

9     def test_pipeline():
10         video_path = r"C:\Users\avidu\Videos\TestLargeVideo.MP4"
11         if not os.path.exists(video_path):
12             print(f"[ERROR] Test video not found at: {video_path}")
13             return
14
15         # Load configuration
16         if os.path.exists("config.json"):
17             with open("config.json", "r") as f:
18                 config = json.load(f)
19         else:
20             config = {}
21
22         # Debug: Trim to first 15 seconds to check 7s-12s rally detection
23         if "indexing" not in config:
24             config["indexing"] = {}
25         config["indexing"]["debug_duration"] = 15.0

```

6. assistant (2026-07-02T03:01:16.544Z)

All core claims verified so far. Let me check the commit history and pytest exclusion.

7. user (2026-07-02T03:01:21.469Z)

```

Exit code 2
8615534 refactor: adopt approved layered directory structure
0f76429 Refactor rally detection to pluggable base and registry, integrate Gemini 2.5 Flash
segmenter, add local fast ffmpeg trimming, and update developer docs
---all 7---
scratch/analyze_motion.py: 72f972e Checkpoint: Add debug trim feature, runner script, and motion
analysis scratch script
scratch/clean_state.py: 6b7d5dd Refactor Gemini prompt to file top, add defensive timestamp
parsing, add compiler logging, and create scratch debug/cleanup tools
scratch/diagnose_rally.py: 0f76429 Refactor rally detection to pluggable base and registry,
integrate Gemini 2.5 Flash segmenter, add local fast ffmpeg trimming, and update developer docs
scratch/gemini_video_debugger.py: 6b7d5dd Refactor Gemini prompt to file top, add defensive
timestamp parsing, add compiler logging, and create scratch debug/cleanup tools
scratch/list_models.py: 0f76429 Refactor rally detection to pluggable base and registry,
integrate Gemini 2.5 Flash segmenter, add local fast ffmpeg trimming, and update developer docs
scratch/test_ai_segmenter.py: 0f76429 Refactor rally detection to pluggable base and registry,
integrate Gemini 2.5 Flash segmenter, add local fast ffmpeg trimming, and update developer docs
scratch/test_thresholds.py: 0f76429 Refactor rally detection to pluggable base and registry,
integrate Gemini 2.5 Flash segmenter, add local fast ffmpeg trimming, and update developer docs
---pytest cfg---
pytest.ini:3:# scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)

```

8. user (2026-07-02T03:01:31.608Z)

[pytest]

When `pytest` is invoked without an explicit path, only collect from tests/. scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.) and should never run as part of the regular test suite.

```

testpaths = tests
---modules exist?---
backend/infrastructure/database.py

```

```

backend/pipeline/segmenters:
__init__.py
__pycache__
---recent commits touching scratch---
8615534 refactor: adopt approved layered directory structure
5257fd5 feat: pipelined YOLO chunk processing & unified Gemini model config

```

6b7d5dd Refactor Gemini prompt to file top, add defensive timestamp parsing, add compiler logging, and create scratch debug/cleanup tools
0f76429 Refactor rally detection to pluggable base and registry, integrate Gemini 2.5 Flash segmenter, add local fast ffmpeg trimming, and update developer docs
72f972e Checkpoint: Add debug trim feature, runner script, and motion analysis scratch script

9. user (2026-07-02T03:02:14.970Z)

Structured output provided successfully